
User Manual

for MPC560XB FLS Driver

Document Number: UM13FLSASR4.0 Rev0003R1.0.1
Rev 3.0





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	11
2.2	Overview.....	12
2.3	About this Manual.....	12
2.4	Acronyms and Definitions.....	13
2.5	Reference List.....	13
Chapter 3		
Driver		
3.1	Requirements.....	15
3.2	Driver Design Summary.....	15
3.3	Deviation from Requirements.....	16
3.4	Runtime Errors.....	20
3.5	Software specification.....	20
3.5.1	Define Reference.....	21
3.5.2	Macros Reference.....	21
3.5.3	Enum Reference.....	21
3.5.4	Function Reference.....	21
3.5.4.1	Function Fls_Cancel.....	21
3.5.4.2	Function Fls_Compare.....	21
3.5.4.3	Function Fls_Erase.....	22
3.5.4.4	Function Fls_GetJobResult.....	23
3.5.4.5	Function Fls_GetStatus.....	24
3.5.4.6	Function Fls_GetVersionInfo.....	24
3.5.4.7	Function Fls_Init.....	25
3.5.4.8	Function Fls_MainFunction.....	25

Section number	Title	Page
3.5.4.9	Function Fls_Read.....	26
3.5.4.10	Function Fls_SetMode.....	27
3.5.4.11	Function Fls_Write.....	27
3.5.5	Structs Reference.....	28
3.5.6	Types Reference.....	28
3.5.7	Variables Reference.....	28
3.6	Symbolic Names DISCLAIMER.....	28

Chapter 4 Tresos Configuration Plug-in

4.1	Configuration elements of Fls.....	31
4.2	Form IMPLEMENTATION_CONFIG_VARIANT.....	31
4.3	Form NonAutosar.....	32
4.3.1	FlsDisableDemReportErrorStatus (NonAutosar).....	32
4.4	Form FlsGeneral.....	33
4.4.1	FlsAcLoadOnJobStart (FlsGeneral).....	33
4.4.2	FlsBaseAddress (FlsGeneral).....	34
4.4.3	FlsCancelApi (FlsGeneral).....	34
4.4.4	FlsCompareApi (FlsGeneral).....	35
4.4.5	FlsDevErrorDetect (FlsGeneral).....	35
4.4.6	FlsDriverIndex (FlsGeneral).....	36
4.4.7	FlsGetJobResultApi (FlsGeneral).....	36
4.4.8	FlsGetStatusApi (FlsGeneral).....	36
4.4.9	FlsSetModeApi (FlsGeneral).....	37
4.4.10	FlsTotalSize (FlsGeneral).....	37
4.4.11	FlsUseInterrupts (FlsGeneral).....	38
4.4.12	FlsVersionInfoApi (FlsGeneral).....	38
4.4.13	FlsDsiHandlerApi (FlsGeneral).....	39
4.4.14	FlsEraseBlankCheck (FlsGeneral).....	39
4.4.15	FlsWriteBlankCheck (FlsGeneral).....	40

Section number	Title	Page
4.4.16	FlsWriteVerifyCheck (FlsGeneral).....	40
4.4.17	FlsMaxEraseBlankCheck (FlsGeneral).....	41
4.4.18	FlsOperationTimeout (FlsGeneral).....	41
4.4.19	FlsAbortTimeout (FlsGeneral).....	42
4.5	Form FlsPublishedInformation.....	42
4.5.1	FlsAcLocationErase (FlsPublishedInformation).....	43
4.5.2	FlsAcLocationWrite (FlsPublishedInformation).....	43
4.5.3	FlsAcSizeErase (FlsPublishedInformation).....	44
4.5.4	FlsAcSizeWrite (FlsPublishedInformation).....	44
4.5.5	FlsEraseTime (FlsPublishedInformation).....	45
4.5.6	FlsErasedValue (FlsPublishedInformation).....	45
4.5.7	FlsExpectedHwId (FlsPublishedInformation).....	46
4.5.8	FlsSpecifiedEraseCycles (FlsPublishedInformation).....	46
4.5.9	FlsWriteTime (FlsPublishedInformation).....	47
4.6	Form CommonPublishedInformation.....	47
4.6.1	ArReleaseMajorVersion (CommonPublishedInformation).....	48
4.6.2	ArReleaseMinorVersion (CommonPublishedInformation).....	48
4.6.3	ArReleaseRevisionVersion (CommonPublishedInformation).....	49
4.6.4	ModuleId (CommonPublishedInformation).....	49
4.6.5	SwMajorVersion (CommonPublishedInformation).....	50
4.6.6	SwMinorVersion (CommonPublishedInformation).....	50
4.6.7	SwPatchVersion (CommonPublishedInformation).....	51
4.6.8	VendorApiInfix (CommonPublishedInformation).....	51
4.6.9	VendorId (CommonPublishedInformation).....	52
4.7	Form FlsConfigSet.....	52
4.7.1	FlsAcErase (FlsConfigSet).....	53
4.7.2	FlsAcWrite (FlsConfigSet).....	54
4.7.3	FlsAcErasePointer (FlsConfigSet).....	54
4.7.4	FlsAcWritePointer (FlsConfigSet).....	54

Section number	Title	Page
4.7.5	FlsCallCycle (FlsConfigSet).....	55
4.7.6	FlsDefaultMode (FlsConfigSet).....	55
4.7.7	FlsACCallback (FlsConfigSet).....	56
4.7.8	FlsJobEndNotification (FlsConfigSet).....	56
4.7.9	FlsJobErrorNotification (FlsConfigSet).....	57
4.7.10	FlsMaxReadFastMode (FlsConfigSet).....	57
4.7.11	FlsMaxReadNormalMode (FlsConfigSet).....	58
4.7.12	FlsMaxWriteFastMode (FlsConfigSet).....	58
4.7.13	FlsMaxWriteNormalMode (FlsConfigSet).....	59
4.7.14	FlsProtection (FlsConfigSet).....	59
4.7.15	Form FlsDemEventParameterRefs.....	60
4.7.15.1	FLS_E_COMPARE_FAILED (FlsDemEventParameterRefs).....	60
4.7.15.2	FLS_E_ERASE_FAILED (FlsDemEventParameterRefs).....	61
4.7.15.3	FLS_E_READ_FAILED (FlsDemEventParameterRefs).....	61
4.7.15.4	FLS_E_UNEXPECTED_FLASH_ID (FlsDemEventParameterRefs).....	61
4.7.15.5	FLS_E_WRITE_FAILED (FlsDemEventParameterRefs).....	62
4.7.16	Form FlsSectorList.....	62
4.7.16.1	Form FlsSector.....	63
4.7.16.1.1	FlsPhysicalSectorUnlock (FlsSector).....	63
4.7.16.1.2	FlsPhysicalSector (FlsSector).....	64
4.7.16.1.3	FlsNumberOfSectors (FlsSector).....	64
4.7.16.1.4	FlsPageSize (FlsSector).....	65
4.7.16.1.5	FlsSectorSize (FlsSector).....	65
4.7.16.1.6	FlsSectorStartaddress (FlsSector).....	65
4.7.16.1.7	FlsProgrammingSize (FlsSector).....	66
4.7.16.1.8	FlsSectorEraseAsynch (FlsSector).....	66
4.7.16.1.9	FlsPageWriteAsynch (FlsSector).....	67

Section number	Title	Page
Chapter 5		
Platform		
5.1	5606B Flash Banks/Arrays, Sectors details	69
5.2	MPC5607B Flash Banks/Arrays, Sectors details	69
5.3	5602D Flash Banks/Arrays, Sectors details	70
5.4	MPC5604B Flash Banks/Arrays, Sectors details	71
5.5	Flash memory physical sectors unlock example	72
Chapter 6		
Other Tressos Configuration		
6.1	Setting up the plugins.....	73
Chapter 7		
Async/sync mode		
7.1	Linear Address.....	75
7.2	Fls Page Size vs Programming Size.....	76
7.3	Async vs Sync.....	76
7.4	Access Code Routine on Job Start.....	77
7.5	Unlocking Fls Sectors.....	77



Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	16-Feb-2012	Cosimo Stornaiuolo	Initial Release for MPC560XB
2.0	14-May-2013	Khanindra Deka	Updated for the MPC560XB RTM 1.0.0
3.0	06-Jun-2014	Guillaume Herry	Updated for the MPC560XB RTM 1.0.1 - Driver Design Summary and configuration updated - Tresos screenshots updated - Global Documentation reviewed



Chapter 2

Introduction

This User Manual describes Freescale Semiconductor AUTOSAR Flash (Fls) for MPC560XB .

AUTOSAR Fls driver configuration parameters and deviations from the specification are described in Fls Driver chapter of this document. AUTOSAR Fls driver requirements and APIs are described in the AUTOSAR Fls driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor .

Table 2-1. MPC560XB Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144
-------------------------	---

Table 2-1. MPC560XB Derivatives

	mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
--	---

All of the above microcontroller devices are collectively named as MPC560XB .

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
FEE	Flash EEPROM Emulation
FLS	Flash
MCU	Micro Controller Unit
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003Fls Driver Software Specification Document.	V4.0 Rev003
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013
6	Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
7	Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
8	Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

Chapter 3 Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.0 Rev0003Fls Driver Software Specification document (See Table [Reference List](#)).

3.2 Driver Design Summary

The FLS driver provides services for reading (**Fls_Read**, **Fls Compare**), writing (**Fls_Write**) and erasing (**Fls_Erase**) flash memory.

Above API will busy the driver, so that no other API can be called in the meantime then a **Fls_MainFunction** should be called cyclically until the driver become IDLE again.

Status of the driver and the job result can be found calling **Fls_GetStatus** and **Fls_JobResult** API.

Flash driver is only to be used by the Flash EEPROM emulation module for writing data. It is not intended to write program code to flash memory in application mode.

The Flash memory physical sectors that are going to be modified by FLS driver (i.e. erase and write operations) have to be unlocked for a successful operation.

It is handled by FLS driver calling **Fls_Init** when **FlsPhysicalSectorUnlock** parameter is enabled at sector configuration level.

It is recommended to configure and unlock only those **FlsPhysicalSector(s)** that are required by upper layer module (FEE), so FLS driver can access only those configured **FlsPhysicalSector(s)** and can not modify rest of Flash address space.

It is also possible to Cancel an ongoing operation (**Fls_Cancel**), if the operation is asynchronous it will abort the ongoing job, if the operation is synchronous it will wait for the end of the HV operation.

In case of reset or power drop during an HV operation, ECC error can be present in Flash location, when reading this location an IVOR exception can be triggered. In this case it is important to use the **Fls_DsiHandler** API into application IVOR handler to inform the driver when this kind of problem is present, more information can be found in the IM document.

Finally **Fls_GetVersion** will give the version of the SW.

It is responsibility of integrator/application to ensure that MCU-wide parameters like voltage supply etc. are according to and in limits specified in MCU documentation. Integrator/application is responsible to implement additional functionality that cancel any on-going erase/write Fls jobs if MCU conditions are not in such limits.

Note: If the microcontroller is in user mode, be sure that the Flash memory controller registers are accessible, for more information please refer to the 'Memory Protection Unit' and 'Register Protection' chapters in the device Reference Manual.

3.3 Deviation from Requirements

The driver deviates from the AUTOSAR Fls Driver software specification in some places.

There are also some additional requirements (on top of requirements detailed in AUTOSAR Fls Driver software specification) which need to be satisfied for correct operation.

Table 3-1. Deviations Status Column Description

Term	Definition
N/A	Not Available
N/T	Not Testable
N/S	Out of Scope
N/I	Not Implemented
N/F	Not Fully Implemented
I/D	Implemented with Deviation

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the driver.

Table 3-2. Driver Deviations Table

SW Requirement ID	Status	Description	Notes
FLS107	N/F	The FLS module shall comply with the following file structure: Figure1 File include structure (see Figure image2.emf)	Not fully compliant
FLS004	N/F	(see Table Table_d2e28879.html)	FLS_E_TIMEOUT not yet implemented; FLS_E_UNEXPECTED_FLASH_ID is applicable only for external FLS driver
FLS319	N/A	The production error code FLS_E_UNEXPECTED_FLASH_ID shall be reported when the expected flash ID is not matched (see FLS144).	Applicable only for external FLS driver
FLS144	N/A	During the initialization of the external flash driver, the FLS module shall check the hardware ID of the external flash device against the corresponding published parameter. <continue>	Applicable only for external FLS driver
FLS272	N/I	If development error detection for the module FLS is enabled: the function FLS_MainFunction shall provide a timeout monitoring for the currently running job, that is it shall supervise the deadline of the read / compare / erase or write job.	Not yet clear how to implement timeout functionality. The hardware has no direct support for this.
FLS359	N/I	If development error detection for the module FLS is enabled: the function FLS_MainFunction shall check, whether the configured maximum erase time <continue>	Not yet clear how to implement timeout functionality. The hardware has no direct support for this.
FLS360	N/I	If development error detection for the module FLS is enabled: the function FLS_MainFunction shall check, whether the expected maximum write time <continue>	Not yet clear how to implement timeout functionality. The hardware has no direct support for this.
FLS361	N/I	The development error code FLS_E_TIMEOUT shall be reported when the timeout supervision of a read, write, erase or compare job failed.	Not yet clear how to implement timeout functionality. The hardware has no direct support for this.
FLS362	N/I	If development error detection for the module FLS is enabled: the function FLS_MainFunction shall check, whether the expected maximum read / compare <continue>	Not yet clear how to implement timeout functionality. The hardware has no direct support for this.
FLS215	N/F	The FLS module's flash access routines shall only disable interrupts and wait for the completion of the erase / write command if necessary (that is if it has to be ensured that no other code is executed in the meantime).	Only RTE plug-in has the ability to enable/disable interrupts. Additionally there is possibility to alter default behaviour and have Erase/Write jobs asynchronous, i.e. FLS_MainFunction function doesn't wait (block) for completion of the erase sector/page write operation(s).

Table continues on the next page...

Table 3-2. Driver Deviations Table (continued)

SW Requirement ID	Status	Description	Notes
FLS217	N/A	The FLS module shall add a device specific base address to the address type Fls_AddressType if necessary.	Unclear concept: device specific base address Not used
FLS014	N/I	The function Fls_Init shall initialize the FLS module (software) and all flash memory relevant registers (hardware) with parameters provided in the given configuration set.	Not Applicable. The FLS module shall check static configuration parameters at configuration or at least at compilation time. See FLS205
FLS208	N/F	The FLS module shall combine all available flash memory areas into one linear address space (denoted by the parameters FlsBaseAddress and FlsTotalSize).	Unclear Pourpose. FlsBaseAddress and FlsTotalSize not used. Impacted requitements are: FLS221, FLS020, FLS226, FLS026, FLS239, FLS097, FLS244, FLS150,
FLS169_conf	N/I	FlsBaseAddress	FlsBaseAddress not used. Unclear purpose
FLS170_conf	N/I	FlsTotalSize	FlsTotalSize not used. Unclear purpose
FLS145	N/I	If possible, e.g. with interrupt controlled implementations, the FLS module shall start the first round of the erase job directly within the function Fls_Erase to reduce overall runtime.	Not applicable. Not supported by hardware. Hardware has no related interrupts.
FLS146	N/I	If possible, e.g. with interrupt controlled implementations, the FLS module shall start the first round of the write job directly within the function Fls_Write to reduce overall runtime.	Not applicable. Not supported by hardware. Hardware has no related interrupts.
FLS247	N/F	If source code for caller and callee of the function Fls_GetVersionInfo is available, the FLS module should realize this function as a macro. The FLS module should define this macro in the module's header file.	The function will be implemented as function, not as a macro.
FLS040	N/F	The function Fls_MainFunction shall only process as much data in one call cycle as statically configured for the current job type (read, write, erase or compare) and the current FLS module's operating mode (normal, fast).	For Erase job not applicable as whole sector(s) is(are) erased.
FLS022	I/D	If development error detection for the module Fls is enabled: After a flash block has been erased, the function Fls_MainFunction shall compare <continue>	Functionality available if both FlsDevErrorDetect and FlsEraseBlankCheck configured to true. If only FlsEraseBlankCheck configured to true the DET error is not reported but Fls job ends with MEMIF_JOB_FAILED.
FLS055	I/D	If development error detection for the module Fls is enabled: Before writing a flash block, the function Fls_MainFunction shall compare <continue>	Functionality available if both FlsDevErrorDetect and FlsWriteBlankCheck configured to true. If only FlsWriteBlankCheck configured to true the DET error is not reported but Fls job ends with MEMIF_JOB_FAILED.
FLS056	I/D	If development error detection for the module Fls is enabled:: After writing a flash	Functionality available if both FlsDevErrorDetect and FlsWriteVerifyCheck configured to true. If only

Table continues on the next page...

Table 3-2. Driver Deviations Table (continued)

SW Requirement ID	Status	Description	Notes
		block, the function Fls_MainFunction shall compare <continue>	FlsWriteVerifyCheck configured to true the DET error is not reported but Fls job ends with MEMIF_JOB_FAILED.
FLS232	N/I	The configuration parameter FlsUseInterrupts shall switch between interrupt and polling controlled job processing if this is supported by the flash memory hardware.	Not applicable. Not supported by hardware.
FLS233	N/I	The FLS module's implementer shall locate the interrupt service routine in Fls_Irq.c.	Not applicable. Not supported by hardware.
FLS234	N/I	If interrupt controlled job processing is supported and enabled with the configuration parameter FlsUseInterrupts, the interrupt service routine shall <continue>	Not applicable. Not supported by hardware.
FLS292_conf	N/I	FlsUseInterrupts	Not applicable. Not supported by hardware.
FLS196	N/I	The function Fls_MainFunction shall at the most issue one sector erase command (to the hardware) in each cycle.	Implementation now erases only one sector per cycle but the HW allows erasing of more physical sectors in parallel (but the final erase time is not reduced).
FLS306_conf	N/I	FlsCallCycle	FlsCallCycle not used, unclear purpose.
FLS280_conf	N/I	FlsNumberOfSectors	FlsNumberOfSectors not used, unclear purpose.
FLS279_conf	N/I	FlsProtection {FLS_PROTECTION} Erase/write protection settings. Only relevant if supported by hardware. (see Table TableConf_d2e40923.html)	FlsProtection: not used. Replaced by Vendor specific parameter (see PR-MCAL-3158)
FLS278_conf	N/F	This value has to correspond to the settings in FLS_PAGE_LIST. The minimum number is defined by the size of one flash page and therefore depends on the underlying flash device.	FLS_PAGE_LIST is unclear defined
FLS277_conf	N/F	This value has to correspond to the settings in FLS_PAGE_LIST. The minimum number is defined by the size of one flash page and therefore depends on the underlying flash device.	FLS_PAGE_LIST is unclear defined
FLS302	N/I	The module's status, mode and the job result shall be made available for debugging (reading). Therefore those variables shall be implemented as global variables.	Support for Debugging not yet implemented.
FLS303	N/I	The type definitions and declarations of all variables which shall be used for debugging shall be given in the modules header file Fls.h.	Support for Debugging not yet implemented.
FLS304	N/I	All variables which shall be used for debugging shall be described in detail in the modules description file.	Support for Debugging not yet implemented.

3.4 Runtime Errors

The driver generates the following DEM errors at runtime.

Table 3-3. Runtime Errors

Function	Error Code	Condition triggering the error
Fls_LLD_MainFunction()	FLS_E_ERASE_FAILED	Async Erase operation failed.
Fls_LLD_MainFunction()	FLS_E_ERASE_FAILED	Async Erase operation failed (in case of interleaved blocks).
Fls_LLD_MainFunction()	FLS_E_WRITE_FAILED	Async Write operation failed.
Fls_LLD_SectorErase()	FLS_E_ERASE_FAILED	Erase operation cannot be executed.
Fls_LLD_SectorErase()	FLS_E_ERASE_FAILED	Erase operation cannot be executed (in case of interleaved blocks).
Fls_LLD_SectorErase()	FLS_E_ERASE_FAILED	Erase operation cannot be executed. Sector is locked, must be unlocked before an HV operation can be set
Fls_LLD_SectorErase()	FLS_E_ERASE_FAILED	Erase operation cannot be executed (in case of interleaved blocks).
Fls_LLD_SectorErase()	FLS_E_ERASE_FAILED	Sync Erase operation failed.
Fls_LLD_SectorErase()	FLS_E_ERASE_FAILED	Sync Erase operation failed (in case of interleaved blocks).
Fls_LLD_SectorErase()	FLS_E_ERASE_FAILED	Sync Erase operation failed (and previous two cases).
Fls_LLD_SectorWrite()	FLS_E_WRITE_FAILED	Write operation cannot be executed.
Fls_LLD_SectorWrite()	FLS_E_WRITE_FAILED	Write operation cannot be executed (in case of interleaved blocks).
Fls_LLD_SectorWrite()	FLS_E_WRITE_FAILED	Write operation cannot be executed. Sector is locked, must be unlocked before an HV operation can be set
Fls_LLD_SectorWrite()	FLS_E_WRITE_FAILED	Write operation cannot be executed (in case of interleaved blocks). Sector is locked, must be unlocked before an HV operation can be set
Fls_LLD_SectorWrite()	FLS_E_WRITE_FAILED	Sync Write operation failed.
Fls_LLD_SectorWrite()	FLS_E_WRITE_FAILED	Sync Write operation failed (in case of interleaved blocks).
Fls_DoJobRead()	FLS_E_READ_FAILED	A non correctable ECC error is present at read location.
Fls_DoJobCompare()	FLS_E_COMPARE_FAILED	A non correctable ECC error is present at read location.

3.5 Software specification

The following sections contains driver software specifications.

3.5.1 Define Reference

Constants supported by the Fls driver.

3.5.2 Macros Reference

Macros supported by the Fls driver.

3.5.3 Enum Reference

Enumeration of all constants supported by the Fls driver.

3.5.4 Function Reference

Functions supported by the Fls driver.

3.5.4.1 Function Fls_Cancel

Cancel an ongoing flash read, write, erase or compare job.

Details:

Abort a running job synchronously so that directly after returning from this function a new job can be started.

Pre: The module must be initialized.

Post: Fls_Cancel changes module status and Fls_JobResult internal variable.

Implements: DFLS02038

Prototype: `void Fls_Cancel(void);`

3.5.4.2 Function Fls_Compare

Compares a flash memory area with an application data buffer.



Figure 3-1. Function Fls_Compare References.

Details:

Starts a compare job asynchronously. The actual job is performed by Fls_MainFunction.

Return: Std_ReturnType.

Pre: The module has to be initialized and not busy.

Post: Fls_Read changes module status and some internal variables (Fls_JobSectorIt, Fls_JobAddrIt, Fls_JobAddrEnd, Fls_JobDataSrcPtr, Fls_Job, Fls_JobResult).

Implements: DFLS02039

Prototype: Std_ReturnType Fls_Compare(Fls_AddressType SourceAddress, const uint8 *TargetAddressPtr, Fls_LengthType Length);

Table 3-4. Fls_Compare Arguments

Type	Name	Direction	Description
Fls_AddressType	SourceAddress	input	Source address in flash memory.
const uint8 *	TargetAddressPtr	input	Pointer to source data buffer.
Fls_LengthType	Length	input	Number of bytes to compare.

Table 3-5. Fls_Compare Return Values

Name	Description
E_OK	Compare command has been accepted.
E_NOT_OK	Compare command has not been accepted.

3.5.4.3 Function Fls_Erase

Erase one or more complete flash sectors.

Details:

Starts an erase job asynchronously. The actual job is performed by the `Fls_MainFunction`.

Return: `Std_ReturnType`.

Pre: The module has to be initialized and not busy.

Post: `Fls_Erase` changes module status and some internal variables (`Fls_JobSectorIt`, `Fls_JobSectorEnd`, `Fls_Job`, `Fls_JobResult`).

Implements: DFLS02045

Prototype: `Std_ReturnType Fls_Erase(Fls_AddressType TargetAddress, Fls_LengthType Length);`

Table 3-6. Fls_Erase Arguments

Type	Name	Direction	Description
<code>Fls_AddressType</code>	<code>TargetAddress</code>	input	Target address in flash memory.
<code>Fls_LengthType</code>	<code>Length</code>	input	Number of bytes to erase.

Table 3-7. Fls_Erase Return Values

Name	Description
<code>E_OK</code>	Erase command has been accepted.
<code>E_NOT_OK</code>	Erase command has not been accepted.

3.5.4.4 Function `Fls_GetJobResult`

Returns the result of the last job.

Details:

Returns synchronously the result of the last job.

Return: `MemIf_JobResultType`.

Implements: DFLS02046

Prototype: `MemIf_JobResultType Fls_GetJobResult(void);`

Table 3-8. Fls_GetJobResult Return Values

Name	Description
<code>MEMIF_JOB_OK</code>	Successfully completed job.

Table continues on the next page...

**Table 3-8. Fls_GetJobResult Return Values
(continued)**

Name	Description
MEMIF_JOB_FAILED	Not successfully completed job.
MEMIF_JOB_PENDING	Still pending job (not yet completed).
MEMIF_JOB_CANCELED	Job has been canceled.
MEMIF_BLOCK_INCONSISTENT	Inconsistent block requested, it may contains corrupted data.
MEMIF_BLOCK_INVALID	Invalid block requested.

3.5.4.5 Function Fls_GetStatus

Returns the FLS module status.

Details:

Returns the FLS module status synchronously.

Return: MemIf_StatusType.

Implements: DFLS02049

Prototype: MemIf_StatusType Fls_GetStatus(void);

Table 3-9. Fls_GetStatus Return Values

Name	Description
MEMIF_UNINIT	Module has not been initialized (yet).
MEMIF_IDLE	Module is currently idle.
MEMIF_BUSY	Module is currently busy.

3.5.4.6 Function Fls_GetVersionInfo

Returns version information about FLS module.

Details:

Version information includes:

- Module Id

- Vendor Id
- Vendor specific version numbers (BSW00407).

Implements: DFLS02050

Prototype: `void Fls_GetVersionInfo(Std_VersionInfoType *VersionInfoPtr);`

Table 3-10. Fls_GetVersionInfo Arguments

Type	Name	Direction	Description
Std_VersionInfoType *	VersionInfoPtr	input, output	Pointer to where to store the version information of this module.

3.5.4.7 Function Fls_Init

The function initializes Fls module.

Details:

The function sets the internal module variables according to given configuration set.

Pre: ConfigPtr must not be NULL_PTR and the module status must not be MEMIF_BUSY.

Implements: DFLS02051

Prototype: `void Fls_Init(const Fls_ConfigType *ConfigPtr);`

Table 3-11. Fls_Init Arguments

Type	Name	Direction	Description
const Fls_ConfigType *	ConfigPtr	input	Pointer to flash driver configuration set.

3.5.4.8 Function Fls_MainFunction

Performs actual flash read, write, erase and compare jobs.

Details:

Bytes number processed per cycle depends by job type (erase, write, read, compare) current FLS module's operating mode (normal, fast) and write, erase Mode of Execution (sync, async).

Pre: The module has to be initialized.

Implements: DFLS02059

Note

This function have to be called ciclically by the Basic Software Module; it will do nothing if there aren't pending job.

Prototype: void Fls_MainFunction(void);

3.5.4.9 Function Fls_Read

Reads from flash memory.

Details:

Starts a read job asynchronously. The actual job is performed by Fls_MainFunction.

Return: MemIf_JobResultType.

Pre: The module has to be initialized and not busy.

Post: Fls_Read changes module status and some internal variables (Fls_JobSectorIt, Fls_JobAddrIt, Fls_JobAddrEnd, Fls_JobDataDestPtr, Fls_Job, Fls_JobResult).

Implements: DFLS02060

Prototype: Std_ReturnType Fls_Read(Fls_AddressType SourceAddress, uint8 *TargetAddressPtr, Fls_LengthType Length);

Table 3-12. Fls_Read Arguments

Type	Name	Direction	Description
Fls_AddressType	SourceAddress	input	Source address in flash memory.
Fls_LengthType	Length	input	Number of bytes to read.
uint8 *	TargetAddressPtr	output	Pointer to target data buffer.

Table 3-13. Fls_Read Return Values

Name	Description
MEMIF_JOB_OK	Successfully completed job.
MEMIF_JOB_FAILED	Not successfully completed job.
MEMIF_JOB_PENDING	Still pending job (not yet completed).

Table continues on the next page...

Table 3-13. Fls_Read Return Values (continued)

Name	Description
MEMIF_JOB_CANCELED	Job has been canceled.
MEMIF_BLOCK_INCONSISTENT	Inconsistent block requested, it may contains corrupted data.
MEMIF_BLOCK_INVALID	Invalid block requested.

3.5.4.10 Function Fls_SetMode

Sets the FLS module's operation mode to the given Mode.

Details:

Every given mode determinates maximum bytes for read/write operations. Every mode has a set of pre-configured values.

Pre: The module has to be initialized and not busy.

Post: Fls_SetMode changes internal variables Fls_MaxRead and Fls_MaxWrite.

Implements: DFLS02061

Prototype: void Fls_SetMode(MemIf_ModeType Mode);

Table 3-14. Fls_SetMode Arguments

Type	Name	Direction	Description
MemIf_ModeType	Mode	input	MEMIF_MODE_FAST or MEMIF_MODE_SLOW.

3.5.4.11 Function Fls_Write

Write one or more complete flash pages to the flash device.

Details:

Starts a write job asynchronously. The actual job is performed by Fls_MainFunction.

Return: Std_ReturnType.

Pre: The module has to be initialized and not busy.

Symbolic Names DISCLAIMER

Post: Fls_Write changes module status and some internal variables (Fls_JobSectorIt, Fls_JobAddrIt, Fls_JobAddrEnd, Fls_JobDataSrcPtr, Fls_Job, Fls_JobResult).

Implements: DFLS02063

Prototype: Std_ReturnType Fls_Write(Fls_AddressType TargetAddress, const uint8 *SourceAddressPtr, Fls_LengthType Length);

Table 3-15. Fls_Write Arguments

Type	Name	Direction	Description
Fls_AddressType	TargetAddress	input	Target address in flash memory.
const uint8 *	SourceAddressPtr	input	Pointer to source data buffer.
Fls_LengthType	Length	input	Number of bytes to write.

Table 3-16. Fls_Write Return Values

Name	Description
E_OK	Write command has been accepted.
E_NOT_OK	Write command has not been accepted.

3.5.5 Structs Reference

Data structures supported by the Fls driver.

3.5.6 Types Reference

Types structures supported by the Fls driver.

3.5.7 Variables Reference

Variables supported by the Fls driver.

3.6 Symbolic Names DISCLAIMER

All containers having the symbolic name tag set as true in the Autosar schema will generate defines like:

```
#define <Container_Short_Name> <Container_ID>
```

For this reason it is forbidden to duplicate the name of such containers across the MCAL configuration, or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the Fls Driver. The most of the parameters are described below.

4.1 Configuration elements of Fls

Included forms :

- IMPLEMENTATION_CONFIG_VARIANT
- NonAutosar
- FlsGeneral
- FlsPublishedInformation
- CommonPublishedInformation
- FlsConfigSet

Table 4-1. Revision table

Revision	Date
4.1.0	2010-12-03

4.2 Form IMPLEMENTATION_CONFIG_VARIANT

VariantPostBuild: Mix of precompile and postbuild time configuration parameters.

If Config Variant = VariantPostBuild, the files Fls_Cfg.h and Fls_PBcfg.c should be used.

If Config Variant = VariantPreCompile, the files Fls_Cfg.h and Fls_Cfg.c should be used.

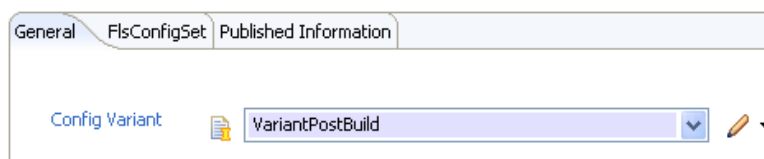


Figure 4-1. Tresos Plugin snapshot for IMPLEMENTATION_CONFIG_VARIANT form.

Table 4-2. Attribute IMPLEMENTATION_CONFIG_VARIANT detailed description

Property	Value
Label	Config Variant
Type	ENUMERATION
Default	VariantPostBuild
Range	VariantPostBuild VariantPreCompile

4.3 Form NonAutosar

NonAutosar

Autosar Requirements:

This container contains the global Non-Autosar configuration parameters of the Fls driver. This container is a MultipleConfigurationContainer, i.e. this container and its sub-containers exist once per configuration set.

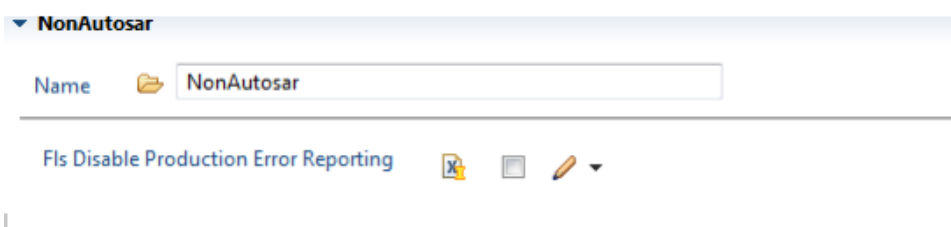


Figure 4-2. Tresos Plugin snapshot for NonAutosar form.

4.3.1 FlsDisableDemReportErrorStatus (NonAutosar)

FlsDisableDemReportErrorStatus

Switches the Diagnostic Error Reporting and Notification OFF.

Table 4-3. Attribute FlsDisableDemReportErrorStatus (NonAutosar) detailed description

Property	Value
Label	Fls Disable Production Error Reporting
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.4 Form FlsGeneral

Container for general parameters of the flash driver. These parameters are always pre-compile.

The screenshot shows the 'Fls General' configuration window. It contains the following parameters and their values:

- Fls Load Access Code On Job Start: ☐
- Fls Base Address (0 -> 4294967295): 0
- Fls Cancel Api: ☒
- Fls Development Error Detect: ☐
- Fls Driver Index (0 -> 254): 0
- Fls Get Job Result Api: ☒
- Fls Set Mode Api: ☒
- Fls Total Size (0 -> 4294967295): 0
- Fls Use Interrupts: ☐
- Fls Dsi Handler Api: ☐
- Fls Write Blank Check: ☒
- Fls Max Erase Blank Check (8 -> 65536): 256
- Fls Operation Timeout (0 -> 2147483647): 2147483647
- Fls Abort Timeout (0 -> 2147483647): 32767
- Fls Compare Api: ☒
- Fls Get Status Api: ☒
- Fls Version Info Api: ☒
- Fls Erase Blank Check: ☒
- Fls Write Verify Check: ☒

Figure 4-3. Tressos Plugin snapshot for FlsGeneral form.

4.4.1 FlsAcLoadOnJobStart (FlsGeneral)

The flash driver shall load the flash access code to RAM whenever an erase or write job is started and unload (overwrite) it after that job has been finished or canceled.

true: Flash access code loaded on job start / unloaded on job end or error.

false: Flash access code not loaded to / unloaded from RAM at all.

Table 4-4. Attribute FlsAcLoadOnJobStart (FlsGeneral) detailed description

Property	Value
Label	Fls Load Access Code On Job Start
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	false

4.4.2 FlsBaseAddress (FlsGeneral)

The flash memory start address (see also FLS118).

FLS169: This parameter defines the lower boundary for read / write / erase and compare jobs.

Note

Not needed / supported by the driver.

Table 4-5. Attribute FlsBaseAddress (FlsGeneral) detailed description

Property	Value
Label	Fls Base Address
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range <=4294967295 >=0

4.4.3 FlsCancelApi (FlsGeneral)

Compile switch to enable and disable the Fls_Cancel function.

true: API supported / function provided.

false: API not supported / function not provided

Table 4-6. Attribute FlsCancelApi (FlsGeneral) detailed description

Property	Value
Label	Fls Cancel Api
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.4.4 FlsCompareApi (FlsGeneral)

Compile switch to enable and disable the Fls_Compare function.

true: API supported / function provided.

false: API not supported / function not provided

Table 4-7. Attribute FlsCompareApi (FlsGeneral) detailed description

Property	Value
Label	Fls Compare Api
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.4.5 FlsDevErrorDetect (FlsGeneral)

Pre-processor switch to enable and disable development error detection (see FLS077).

true: Development error detection enabled.

false: Development error detection disabled.

Table 4-8. Attribute FlsDevErrorDetect (FlsGeneral) detailed description

Property	Value
Label	Fls Development Error Detect
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.4.6 FlsDriverIndex (FlsGeneral)

Index of the driver, used by FEE.

Table 4-9. Attribute FlsDriverIndex (FlsGeneral) detailed description

Property	Value
Label	Fls Driver Index
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	true
Default	0
Invalid	Range <=254 >=0

4.4.7 FlsGetJobResultApi (FlsGeneral)

Compile switch to enable and disable the Fls_GetJobResult function.

true: API supported / function provided.

false: API not supported / function not provided

Table 4-10. Attribute FlsGetJobResultApi (FlsGeneral) detailed description

Property	Value
Label	Fls Get Job Result Api
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.4.8 FlsGetStatusApi (FlsGeneral)

Compile switch to enable and disable the Fls_GetStatus function.

true: API supported / function provided.

false: API not supported / function not provided

Table 4-11. Attribute FlsGetStatusApi (FlsGeneral) detailed description

Property	Value
Label	Fls Get Status Api
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.4.9 FlsSetModeApi (FlsGeneral)

Compile switch to enable and disable the Fls_SetMode function.

true: API supported / function provided.

false: API not supported / function not provided

Table 4-12. Attribute FlsSetModeApi (FlsGeneral) detailed description

Property	Value
Label	Fls Set Mode Api
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.4.10 FlsTotalSize (FlsGeneral)

The total amount of flash memory in bytes (see also FLS118). FLS170: This parameter in conjunction with FLS_BASE_ADDRESS defines the upper boundary for read / write / erase and compare jobs.

Note

Not needed / supported by the driver.

Table 4-13. Attribute FlsTotalSize (FlsGeneral) detailed description

Property	Value
Label	Fls Total Size
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC

Table continues on the next page...

Table 4-13. Attribute FlsTotalSize (FlsGeneral) detailed description (continued)

Property	Value
Symbolic Name	false
Default	0
Invalid	Range <=4294967295 >=0

4.4.11 FlsUseInterrupts (FlsGeneral)

Job processing triggered by hardware interrupt.

true: Job processing triggered by interrupt (hardware controlled)

false: Job processing not triggered by interrupt (software controlled)

Note

Not supported by hardware.

Table 4-14. Attribute FlsUseInterrupts (FlsGeneral) detailed description

Property	Value
Label	Fls Use Interrupts
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	false

4.4.12 FlsVersionInfoApi (FlsGeneral)

Pre-processor switch to enable / disable the API to read out the modules version information.

true: Version info API enabled.

false: Version info API disabled.

Table 4-15. Attribute FlsVersionInfoApi (FlsGeneral) detailed description

Property	Value
Label	Fls Version Info Api

Table continues on the next page...

Table 4-15. Attribute FlsVersionInfoApi (FlsGeneral) detailed description (continued)

Property	Value
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.4.13 FlsDsiHandlerApi (FlsGeneral)

Pre-processor switch to enable / disable the API to report data storage (ECC) errors to the flash driver.

true: Data storage handler API enabled.

false: Data storage handler API disabled.

Note

Vendor specific parameter

Table 4-16. Attribute FlsDsiHandlerApi (FlsGeneral) detailed description

Property	Value
Label	Fls Dsi Handler Api
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.4.14 FlsEraseBlankCheck (FlsGeneral)

Pre-processor switch to enable / disable the erase blank check. After a flash block has been erased, the erase blank check compares the contents of the addressed memory area against the value of an erased flash cell to check that the block has been completely erased.

true: Erase blank check enabled.

false: Erase blank check disabled.

Note

Vendor specific parameter

Table 4-17. Attribute FlsEraseBlankCheck (FlsGeneral) detailed description

Property	Value
Label	Fls Erase Blank Check
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	true

4.4.15 FlsWriteBlankCheck (FlsGeneral)

Pre-processor switch to enable / disable the write blank check. Before writing a flash block, the write blank check compares the contents of the addressed memory area against the value of an erased flash cell to check that the block has been completely erased.

true: Write blank check enabled.

false: Write blank check disabled.

Note

Vendor specific parameter

Table 4-18. Attribute FlsWriteBlankCheck (FlsGeneral) detailed description

Property	Value
Label	Fls Write Blank Check
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	true

4.4.16 FlsWriteVerifyCheck (FlsGeneral)

Pre-processor switch to enable / disable the write verify check. After writing a flash block, the write verify check compares the contents of the reprogrammed memory area against the contents of the provided application buffer to check that the block has been completely reprogrammed.

true: Write verify check enabled.

false: Write verify check disabled.

Note

Vendor specific parameter

Table 4-19. Attribute FlsWriteVerifyCheck (FlsGeneral) detailed description

Property	Value
Label	Fls Write Verify Check
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	true

4.4.17 FlsMaxEraseBlankCheck (FlsGeneral)

The maximum number of bytes to blank check in one cycle of the flash driver's job processing function. Affects only the flash blocks that have enabled asynchronous execution of the erase job (FlsSectorEraseAsynch=true).

Note

Vendor specific parameter

Table 4-20. Attribute FlsMaxEraseBlankCheck (FlsGeneral) detailed description

Property	Value
Label	Fls Max Erase Blank Check
Type	INTEGER
Origin	Custom
Symbolic Name	false
Default	256
Invalid	Range <=65536 >=8

4.4.18 FlsOperationTimeout (FlsGeneral)

Fls Operation Timeout is the timeout value for Write and Erase Operation (also used for resumed operation)

Note

Vendor specific parameter

Table 4-21. Attribute FlsOperationTimeout (FlsGeneral) detailed description

Property	Value
Label	Fls Operation Timeout
Type	INTEGER
Origin	Custom
Symbolic Name	false
Default	2147483647
Invalid	Range <=2147483647 >=0

4.4.19 FlsAbortTimeout (FlsGeneral)

Fls Abort Timeout is the timeout value for Abort operation

Note

Vendor specific parameter

Table 4-22. Attribute FlsAbortTimeout (FlsGeneral) detailed description

Property	Value
Label	Fls Abort Timeout
Type	INTEGER
Origin	Custom
Symbolic Name	false
Default	32767
Invalid	Range <=2147483647 >=0

4.5 Form FlsPublishedInformation

Additional published parameters not covered by CommonPublishedInformation container. Note that these parameters do not have any configuration class setting, since they are published information.

Name  FlsPublishedInformation

Fls Access Code Location Erase	 0
Fls Access Code Location Write	 0
Fls Access Code Size Erase	 84
Fls Access Code Size Write	 84
Fls Erase Time	 5.0
Fls Erased Value	 4294967295
Fls Expected Hw Id	 0
Fls Specified Erase Cycles	 100000
Fls Write Time	 5.0E-4

Figure 4-4. Tresos Plugin snapshot for FlsPublishedInformation form.

4.5.1 FlsAcLocationErase (FlsPublishedInformation)

Position in RAM, to which the erase flash access code has to be loaded. Only relevant if the erase flash access code is not position independent. If this information is not provided it is assumed that the erase flash access code is position independent and that therefore the RAM position can be freely configured.

Table 4-23. Attribute FlsAcLocationErase (FlsPublishedInformation) detailed description

Property	Value
Label	Fls Access Code Location Erase
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range <=4294967295 >=0

4.5.2 FlsAcLocationWrite (FlsPublishedInformation)

Position in RAM, to which the write flash access code has to be loaded. Only relevant if the write flash access code is not position independent. If this information is not provided it is assumed that the write flash access code is position independent and that therefore the RAM position can be freely configured.

Table 4-24. Attribute FlsAcLocationWrite (FlsPublishedInformation) detailed description

Property	Value
Label	Fls Access Code Location Write
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range <=4294967295 >=0

4.5.3 FlsAcSizeErase (FlsPublishedInformation)

Number of bytes in RAM needed for the erase flash access code.

Table 4-25. Attribute FlsAcSizeErase (FlsPublishedInformation) detailed description

Property	Value
Label	Fls Accessee Code Size Erase
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	84
Invalid	Range <=4294967295 >=0

4.5.4 FlsAcSizeWrite (FlsPublishedInformation)

Number of bytes in RAM needed for the write flash access code.

Table 4-26. Attribute FlsAcSizeWrite (FlsPublishedInformation) detailed description

Property	Value
Label	Fls Accesse Code Size Write
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	84
Invalid	Range <=4294967295 >=0

4.5.5 FlsEraseTime (FlsPublishedInformation)

Maximum time to erase one complete flash sector [sec].

Note

This value can be found on DS as the maximum erase time occurs after the specified number of program/erase cycles .

Table 4-27. Attribute FlsEraseTime (FlsPublishedInformation) detailed description

Property	Value
Label	Fls Erase Time
Type	FLOAT_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	5.0
Invalid	Range <=5.0 >=0

4.5.6 FlsErasedValue (FlsPublishedInformation)

The contents of an erased flash memory cell.

Table 4-28. Attribute FlsErasedValue (FlsPublishedInformation) detailed description

Property	Value
Label	Fls Erased Value
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	4294967295
Invalid	Range <=4294967295 >=0

4.5.7 FlsExpectedHwId (FlsPublishedInformation)

Unique identifier of the hardware device that is expected by this driver (the device for which this driver has been implemented). Only relevant for external flash drivers.

Table 4-29. Attribute FlsExpectedHwId (FlsPublishedInformation) detailed description

Property	Value
Label	Fls Expected Hw Id
Type	STRING_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0

4.5.8 FlsSpecifiedEraseCycles (FlsPublishedInformation)

Number of erase cycles specified for the flash device (usually given in the device data sheet). FLS198: If the number of specified erase cycles depends on the operating environment (temperature, voltage, ...) during reprogramming of the flash device, the minimum number for which a data retention of at least 15 years over the temperature range from -40C .. +125C can be guaranteed shall be given.

Note

If there are different numbers of specified erase cycles for different flash sectors of the device this parameter has to be extended to a parameter list (similar to the sector list above).

Table 4-30. Attribute FlsSpecifiedEraseCycles (FlsPublishedInformation) detailed description

Property	Value
Label	Fls Specified Erase Cycles
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	100000
Invalid	Range <=4294967295 >=0

4.5.9 FlsWriteTime (FlsPublishedInformation)

Maximum time to program one complete flash page [sec].

Table 4-31. Attribute FlsWriteTime (FlsPublishedInformation) detailed description

Property	Value
Label	Fls Write Time
Type	FLOAT_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0.0005
Invalid	Range <=0.0005 >=0

4.6 Form CommonPublishedInformation

CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Common Published Information		
AUTOSAR Major Version	4	
AUTOSAR Minor Version	0	
AUTOSAR Patch Version	3	
Numeric Module ID	92	
Software Major Version	1	
Software Minor Version	0	
Software Patch Version	1	
Vendor Api Infix		
Vendor ID	43	

Figure 4-5. Tresos Plugin snapshot for CommonPublishedInformation form.

4.6.1 ArReleaseMajorVersion (CommonPublishedInformation)

AUTOSAR Major Version

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 4-32. Attribute ArReleaseMajorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	AUTOSAR Major Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	4
Invalid	Range >=4 <=4

4.6.2 ArReleaseMinorVersion (CommonPublishedInformation)

AUTOSAR Minor Version

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 4-33. Attribute ArReleaseMinorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	AUTOSAR Minor Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	0
Invalid	Range >=0 <=0

4.6.3 ArReleaseRevisionVersion (CommonPublishedInformation)

AUTOSAR Patch Version

Patch version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 4-34. Attribute ArReleaseRevisionVersion (CommonPublishedInformation) detailed description

Property	Value
Label	AUTOSAR Patch Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	3
Invalid	Range >=3 <=3

4.6.4 ModuleId (CommonPublishedInformation)

Module ID

Module ID of this module.

Table 4-35. Attribute ModuleId (CommonPublishedInformation) detailed description

Property	Value
Label	Numeric Module ID
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	92
Invalid	Range >=92 <=92

4.6.5 SwMajorVersion (CommonPublishedInformation)

Software Major Version

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 4-36. Attribute SwMajorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	Software Major Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	1
Invalid	Range >=1 <=1

4.6.6 SwMinorVersion (CommonPublishedInformation)

Software Minor Version

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 4-37. Attribute SwMinorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	Software Minor Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	0
Invalid	Range >=0 <=0

4.6.7 SwPatchVersion (CommonPublishedInformation)

Software Patch Version

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 4-38. Attribute SwPatchVersion (CommonPublishedInformation) detailed description

Property	Value
Label	Software Patch Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	1
Invalid	Range >=1 <=1

4.6.8 VendorApiInfix (CommonPublishedInformation)

Vendor Api Infix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name. This parameter is used to specify the vendor specific name. In total, the implementation

specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix><Api name from SWS>. E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write. This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Table 4-39. Attribute VendorApiInfix (CommonPublishedInformation) detailed description

Property	Value
Label	Vendor Api Infix
Type	STRING_LABEL
Origin	Custom
Symbolic Name	false
Default	
Enable	false

4.6.9 VendorId (CommonPublishedInformation)

Vendor ID

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Table 4-40. Attribute VendorId (CommonPublishedInformation) detailed description

Property	Value
Label	Vendor ID
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	43
Invalid	Range >=43 <=43

4.7 Form FlsConfigSet

Container for runtime configuration parameters of the flash driver.

Implementation Type: Fls_ConfigType.

Included forms :

- [Form FlsDemEventParameterRefs](#)
- [Form FlsSectorList](#)

Property	Value
Fls Access Code Erase (0 -> 4294967295)	1073751296
Fls Access Code Write (0 -> 4294967295)	1073751296
Fls Access Code Erase Pointer	NULL_PTR
Fls Access Code Write Pointer	NULL_PTR
Fls Call Cycle (0 -> 1)	0.0
Fls Default Mode	MEMIF_MODE_SLOW
Fls AC Callback	Fls_AC_Callback_Test
Fls Job End Notification	Fls_JobEndNotif
Fls Job Error Notification	Fls_JobErrorNotif
Fls Max Read FastMode (0 -> 4294967295)	48
Fls Max Read Normal Mode (0 -> 4294967295)	24
Fls Max Write Fast Mode (0 -> 4294967295)	16
Fls Max Write Normal Mode (0 -> 4294967295)	8
Fls Protection (0 -> 4294967295)	0

Figure 4-6. Tresos Plugin snapshot for FlsConfigSet form.

4.7.1 FlsAcErase (FlsConfigSet)

Address offset in RAM to which the erase flash access code shall be loaded. Used as function pointer to access the erase flash access code.

Note: To use Fls Access Code Erase be sure Fls Access Code Erase Pointer is NULL or NULL_PTR.

Table 4-41. Attribute FlsAcErase (FlsConfigSet) detailed description

Property	Value
Label	Fls Access Code Erase
Type	INTEGER
Origin	AUTOSAR_ECUC

Table continues on the next page...

Table 4-41. Attribute FlsAcErase (FlsConfigSet) detailed description (continued)

Property	Value
Symbolic Name	false
Invalid	Range <=4294967295 >=0

4.7.2 FlsAcWrite (FlsConfigSet)

Address offset in RAM to which the write flash access code shall be loaded. Used as function pointer to access the write flash access code.

Note: To use Fls Access Code Write be sure Fls Access Code Write Pointer is NULL or NULL_PTR.

Table 4-42. Attribute FlsAcWrite (FlsConfigSet) detailed description

Property	Value
Label	Fls Access Code Write
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Invalid	Range <=4294967295 >=0

4.7.3 FlsAcErasePointer (FlsConfigSet)

Pointer in RAM to which the erase flash access code shall be loaded.

Table 4-43. Attribute FlsAcErasePointer (FlsConfigSet) detailed description

Property	Value
Label	Fls Access Code Erase Pointer
Type	FUNCTION-NAME
Origin	Custom
Symbolic Name	false
Default	NULL_PTR

4.7.4 FlsAcWritePointer (FlsConfigSet)

Pointer in RAM to which the write flash access code shall be loaded. Used as function pointer to access the write flash access code.

Table 4-44. Attribute FlsAcWritePointer (FlsConfigSet) detailed description

Property	Value
Label	Fls Access Code Write Pointer
Type	FUNCTION-NAME
Origin	Custom
Symbolic Name	false
Default	NULL_PTR

4.7.5 FlsCallCycle (FlsConfigSet)

Cycle time of calls of the flash driver main function

Note

Not supported by the driver.

Table 4-45. Attribute FlsCallCycle (FlsConfigSet) detailed description

Property	Value
Label	Fls Call Cycle
Type	FLOAT_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0.0
Invalid	Range <=1.0 >=0.0

4.7.6 FlsDefaultMode (FlsConfigSet)

This parameter is the default FLS device mode after initialization.

Table 4-46. Attribute FlsDefaultMode (FlsConfigSet) detailed description

Property	Value
Label	Fls Default Mode

Table continues on the next page...

Table 4-46. Attribute FlsDefaultMode (FlsConfigSet) detailed description (continued)

Property	Value
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	MEMIF_MODE_SLOW
Range	MEMIF_MODE_FAST MEMIF_MODE_SLOW

4.7.7 FlsACCallback (FlsConfigSet)

When enabled, mapped to the Access Code Callback provided by some upper layer module, typically the Wdg module.

Otherwise set to NULL_PTR if disabled or deleted

Table 4-47. Attribute FlsACCallback (FlsConfigSet) detailed description

Property	Value
Label	Fls AC Callback
Type	FUNCTION-NAME
Origin	Custom
Symbolic Name	false
Default	Fls_AC_Callback
Enable	false

4.7.8 FlsJobEndNotification (FlsConfigSet)

When enabled, mapped to the job end notification routine provided by some upper layer module, typically the Fee module.

Otherwise set to NULL_PTR if disabled or deleted

Table 4-48. Attribute FlsJobEndNotification (FlsConfigSet) detailed description

Property	Value
Label	Fls Job End Notification
Type	FUNCTION-NAME
Origin	AUTOSAR_ECUC

Table continues on the next page...

Table 4-48. Attribute FlsJobEndNotification (FlsConfigSet) detailed description (continued)

Property	Value
Symbolic Name	false
Default	JobEndNotification
Enable	false

4.7.9 FlsJobErrorNotification (FlsConfigSet)

When enabled, mapped to the job error notification routine provided by some upper layer module, typically the Fee module.

Otherwise set to NULL_PTR if disabled or deleted

Table 4-49. Attribute FlsJobErrorNotification (FlsConfigSet) detailed description

Property	Value
Label	Fls Job Error Notification
Type	FUNCTION-NAME
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	JobErrorNotification
Enable	false

4.7.10 FlsMaxReadFastMode (FlsConfigSet)

The maximum number of bytes to read or compare in one cycle of the flash driver's job processing function in fast mode.

Note: This is a generic parameter which better applies to external flash drivers. As this driver accesses internal flash, where read accesses can be made very fast (only a few wait-states are added in general), it is recommended not to specify a small number.

Table 4-50. Attribute FlsMaxReadFastMode (FlsConfigSet) detailed description

Property	Value
Label	Fls Max Read FastMode
Type	INTEGER

Table continues on the next page...

Table 4-50. Attribute FlsMaxReadFastMode (FlsConfigSet) detailed description (continued)

Property	Value
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	1048576
Invalid	Range <=4294967295 >=0

4.7.11 FlsMaxReadNormalMode (FlsConfigSet)

The maximum number of bytes to read or compare in one cycle of the flash driver's job processing function in normal mode.

Note: This is a generic parameter which better applies to external flash drivers. As this driver accesses internal flash, where read accesses can be made very fast (only a few wait-states are added in general), it is recommended not to specify a small number.

Table 4-51. Attribute FlsMaxReadNormalMode (FlsConfigSet) detailed description

Property	Value
Label	Fls Max Read Normal Mode
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	1024
Invalid	Range <=4294967295 >=0

4.7.12 FlsMaxWriteFastMode (FlsConfigSet)

The maximum number of bytes to write in one cycle of the flash driver's job processing function in fast mode.

Table 4-52. Attribute FlsMaxWriteFastMode (FlsConfigSet) detailed description

Property	Value
Label	Fls Max Write Fast Mode
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	256
Invalid	Range <=4294967295 >=0

4.7.13 FlsMaxWriteNormalMode (FlsConfigSet)

The maximum number of bytes to write in one cycle of the flash driver's job processing function in normal mode.

Table 4-53. Attribute FlsMaxWriteNormalMode (FlsConfigSet) detailed description

Property	Value
Label	Fls Max Write Normal Mode
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	8
Invalid	Range <=4294967295 >=0

4.7.14 FlsProtection (FlsConfigSet)

Erase/write protection settings. Only relevant if supported by hardware.

Note

Not supported by the driver.

Table 4-54. Attribute FlsProtection (FlsConfigSet) detailed description

Property	Value
Label	Fls Protection
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range <=4294967295 >=0

4.7.15 Form FlsDemEventParameterRefs

Container for the references to DemEventParameter elements which shall be invoked using the Dem_ReportErrorStatus API in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.

Is included by form : [Form FlsConfigSet](#)

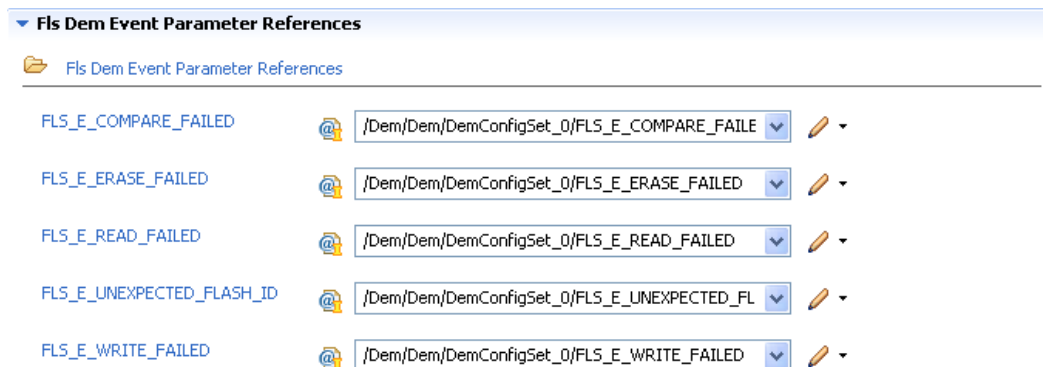


Figure 4-7. Tresos Plugin snapshot for FlsDemEventParameterRefs form.

4.7.15.1 FLS_E_COMPARE_FAILED (FlsDemEventParameterRefs)

Reference to the DemEventParameter which shall be issued when the error "Flash compare failed (HW)" has occurred.

**Table 4-55. Attribute FLS_E_COMPARE_FAILED
(FlsDemEventParameterRefs) detailed description**

Property	Value
Type	SYMBOLIC-NAME-REFERENCE
Origin	AUTOSAR_ECUC
Enable	true

4.7.15.2 FLS_E_ERASE_FAILED (FlsDemEventParameterRefs)

Reference to the DemEventParameter which shall be issued when the error "Flash erase failed (HW)" has occurred.

**Table 4-56. Attribute FLS_E_ERASE_FAILED
(FlsDemEventParameterRefs) detailed description**

Property	Value
Type	SYMBOLIC-NAME-REFERENCE
Origin	AUTOSAR_ECUC
Enable	true

4.7.15.3 FLS_E_READ_FAILED (FlsDemEventParameterRefs)

Reference to the DemEventParameter which shall be issued when the error "Flash read failed (HW)" has occurred.

**Table 4-57. Attribute FLS_E_READ_FAILED
(FlsDemEventParameterRefs) detailed description**

Property	Value
Type	SYMBOLIC-NAME-REFERENCE
Origin	AUTOSAR_ECUC
Enable	true

4.7.15.4 FLS_E_UNEXPECTED_FLASH_ID (FlsDemEventParameterRefs)

Reference to the DemEventParameter which shall be issued when the error "Expected hardware ID not matched" has occurred.

**Table 4-58. Attribute FLS_E_UNEXPECTED_FLASH_ID
(FlsDemEventParameterRefs) detailed description**

Property	Value
Type	SYMBOLIC-NAME-REFERENCE
Origin	AUTOSAR_ECUC
Enable	true

4.7.15.5 FLS_E_WRITE_FAILED (FlsDemEventParameterRefs)

Reference to the DemEventParameter which shall be issued when the error "Flash write failed (HW)" has occurred.

**Table 4-59. Attribute FLS_E_WRITE_FAILED
(FlsDemEventParameterRefs) detailed description**

Property	Value
Type	SYMBOLIC-NAME-REFERENCE
Origin	AUTOSAR_ECUC
Enable	true

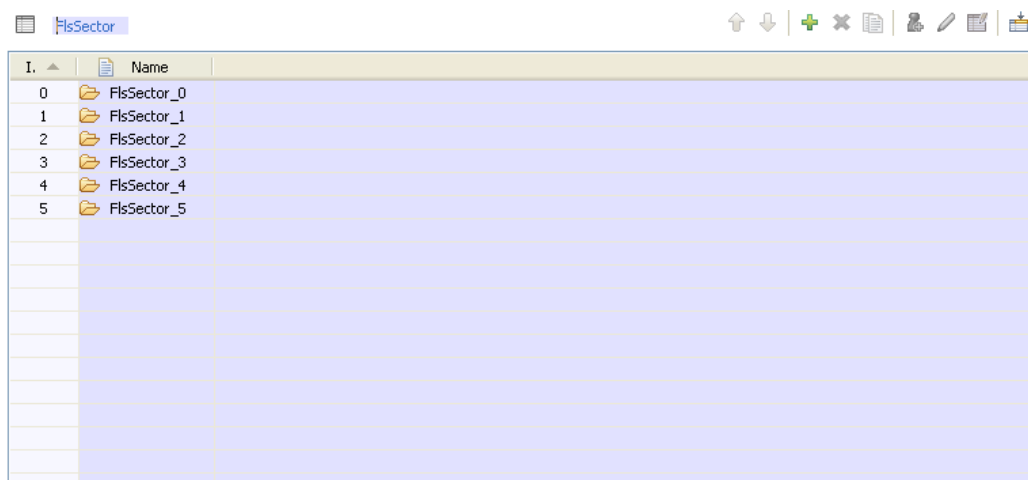
4.7.16 Form FlsSectorList

List of flashable sectors and pages.

Is included by form : [Form FlsConfigSet](#)

Included forms :

- [Form FlsSector](#)



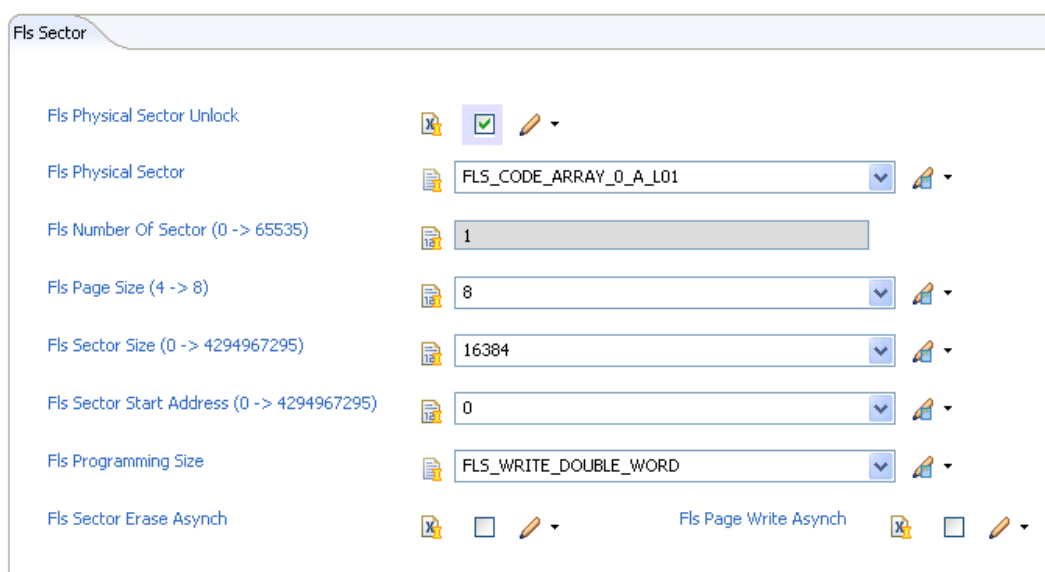
I.		Name
0	📁	FlsSector_0
1	📁	FlsSector_1
2	📁	FlsSector_2
3	📁	FlsSector_3
4	📁	FlsSector_4
5	📁	FlsSector_5

Figure 4-8. Tresos Plugin snapshot for FlsSectorList form.

4.7.16.1 Form FlsSector

Configuration description of a flashable sector

Is included by form : [Form FlsSectorList](#)



Fls Sector

Fls Physical Sector Unlock ☒

Fls Physical Sector

Fls Number Of Sector (0 -> 65535)

Fls Page Size (4 -> 8)

Fls Sector Size (0 -> 4294967295)

Fls Sector Start Address (0 -> 4294967295)

Fls Programming Size

Fls Sector Erase Asynch ☐ Fls Page Write Asynch ☐

Figure 4-9. Tresos Plugin snapshot for FlsSector form.

4.7.16.1.1 FlsPhysicalSectorUnlock (FlsSector)

Fls_Init ensures unlock modify operation for this Flash Physical Sector, it is not possible to lock until a new reset.

Note

Vendor specific parameter

Table 4-60. Attribute FlsPhysicalSectorUnlock (FlsSector) detailed description

Property	Value
Label	Fls Physical Sector Unlock
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	true

4.7.16.1.2 FlsPhysicalSector (FlsSector)

Physical flash device sector.

Note

Vendor specific parameter

Table 4-61. Attribute FlsPhysicalSector (FlsSector) detailed description

Property	Value
Label	Fls Physical Sector
Type	ENUMERATION
Origin	Custom
Symbolic Name	false

4.7.16.1.3 FlsNumberOfSectors (FlsSector)

Number of continuous sectors with the above characteristics.

Note

Not supported by the driver.

Table 4-62. Attribute FlsNumberOfSectors (FlsSector) detailed description

Property	Value
Label	Fls Number Of Sector
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false

Table continues on the next page...

Table 4-62. Attribute FlsNumberOfSectors (FlsSector) detailed description (continued)

Property	Value
Default	1
Invalid	Range <=65535 >=0

4.7.16.1.4 FlsPageSize (FlsSector)

Size of one page of this sector. Implementation Type: Fls_LengthType.

- For Code Flash page size is 8 byte
- For Data Flash page size is 8 byte
- For Data Flash Optimized (DFO) page size is 4 byte

Table 4-63. Attribute FlsPageSize (FlsSector) detailed description

Property	Value
Label	Fls Page Size
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Invalid	Range <=8 >=4

4.7.16.1.5 FlsSectorSize (FlsSector)

Size of this sector.

Implementation Type: Fls_LengthType.

Table 4-64. Attribute FlsSectorSize (FlsSector) detailed description

Property	Value
Label	Fls Sector Size
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Invalid	Range <=4294967295 >=0

4.7.16.1.6 FlsSectorStartaddress (FlsSector)

Start address of this sector.

Implementation Type: Fls_AddressType.

Table 4-65. Attribute FlsSectorStartaddress (FlsSector) detailed description

Property	Value
Label	Fls Sector Start Address
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Invalid	Range <=4294967295 >=0

4.7.16.1.7 FlsProgrammingSize (FlsSector)

This is the size to program Flash memory during one High voltage operation.

Table 4-66. Attribute FlsProgrammingSize (FlsSector) detailed description

Property	Value
Label	Fls Programming Size
Type	ENUMERATION
Origin	Custom
Symbolic Name	false

4.7.16.1.8 FlsSectorEraseAsynch (FlsSector)

Enable asynchronous execution of the erase job in the Fls_MainFunction function which doesn't wait (block) for completion of the sector erase operation. The flash driver doesn't use the erase access code to the erase flash sector in asynchronous mode so it can be used only on flash sectors which belong to flash array different from flash array the application is executing from.

Note

Vendor specific parameter

Table 4-67. Attribute FlsSectorEraseAsynch (FlsSector) detailed description

Property	Value
Label	Fls Sector Erase Asynch
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.7.16.1.9 FlsPageWriteAsynch (FlsSector)

Enable asynchronous execution of the write job in the Fls_MainFunction function which doesn't wait (block) for completion of the page write operation(s). The flash driver doesn't use the write access code to the write flash page(s) in asynchronous mode so it can be used only on flash sectors which belong to flash array different from flash array the application is executing from.

Note

Vendor specific parameter

Table 4-68. Attribute FlsPageWriteAsynch (FlsSector) detailed description

Property	Value
Label	Fls Page Write Asynch
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

Chapter 5 Platform

5.1 5606B Flash Banks/Arrays, Sectors details

The Flash memory comprises a platform Flash controller interface and two Flash memory arrays: one array of 256K for code (Code Flash) and one array of 64 KB for Data Flash.

Table 5-1. Sectors details Bolero 256K

Type	Array	Sector Name	Sector Size (KB)
CODE_ARRAY (Low and Middle Address Space)	0	L0	32
	0	L1	16
	0	L2	16
	0	L3	32
	0	L4	32
	0	L5	128
CODE_TEST	0	T0	16
DFO_ARRAY	1	L0	16
	1	L1	16
	1	L2	16
	1	L3	16
DATA_TEST	1	T1	16

5.2 MPC5607B Flash Banks/Arrays, Sectors details

The Flash memory comprises a platform Flash controller interface and two Flash memory arrays: one array of 512 KB for code (Code Flash) and one array of 64 KB for Data Flash.

Table 5-2. Sectors details Bolero 512K

Type	Array	Sector Name	Sector Size (KB)
CODE_ARRAY (Low and Middle Address Space)	0	L0	32
	0	L1	16
	0	L2	16
	0	L3	32
	0	L4	32
	0	L5	128
	0	M0	128
	0	M1	128
CODE_SHADOW	0	S0	16
CODE_TEST	0	T0	16
DATA_ARRAY	1	L0	16
	1	L1	16
	1	L2	16
	1	L3	16
DATA_TEST	1	T1	16

5.3 5602D Flash Banks/Arrays, Sectors details

The Flash memory comprises a platform Flash controller interface and two Flash memory arrays: one array of 1M for code (Code Flash) and one array of 64 KB for Data Flash.

Table 5-3. Sectors details Bolero 1M

Type	Array	Sector Name	Sector Size (KB)
CODE_ARRAY (Low and Middle Address Space)	0	L0	32
	0	L1	16
	0	L2	16
	0	L3	32
	0	L4	32
	0	L5	128
	0	M0	128
	0	M1	128
	0	H0	128
	0	H1	128
	0	H2	128
	0	H3	128
	0	S0	16
CODE_SHADOW	0	S0	16

Table continues on the next page...

Table 5-3. Sectors details Bolero 1M (continued)

CODE_TEST	0	T0	16
DATA_ARRAY	1	L0	16
	1	L1	16
	1	L2	16
	1	L3	16
DATA_TEST	1	T1	16

5.4 MPC5604B Flash Banks/Arrays, Sectors details

The Flash memory comprises a platform Flash controller interface and two Flash memory arrays: one array of 1.5M for code (Code Flash) and one array of 64 KB for Data Flash.

Table 5-4. Sectors details Bolero 1.5M

Type	Array	Sector Name	Sector Size (KB)
CODE_ARRAY (Low and Middle Address Space)	0	L0	32
	0	L1	16
	0	L2	16
	0	L3	32
	0	L4	32
	0	L5	128
	0	M0	128
	0	M1	128
	0	H0	128
	0	H1	128
	0	H2	128
	0	H3	128
	0	H4	128
	0	H5	128
	0	H6	128
	0	H7	128
CODE_SHADOW	0	S0	16
CODE_TEST	0	T0	16
DATA_ARRAY	1	L0	16
	1	L1	16
	1	L2	16
	1	L3	16
DATA_TEST	1	T1	16

5.5 Flash memory physical sectors unlock example

Below is the code example that unlocks all Flash memory physical sectors.

```
void Fls_SectorsUnlock(void)
{
    /* FLASHMEM0_CF0_A unprotect/unlock */
    *((volatile uint32*)(0xC3F88004)) = 0xA1A11111; /* FLASH_LML */
    *((volatile uint32*)(0xC3F88004)) = 0x00000000; /* FLASH_LML */
    *((volatile uint32*)(0xC3F88008)) = 0xB2B22222; /* FLASH_HBL */
    *((volatile uint32*)(0xC3F88008)) = 0x00000000; /* FLASH_HBL */
    *((volatile uint32*)(0xC3F8800C)) = 0xC3C33333; /* FLASH_SLL */
    *((volatile uint32*)(0xC3F8800C)) = 0x00000000; /* FLASH_SLL */

    /* FLASHMEM0_DF0_A unprotect/unlock */
    *((volatile uint32*)(0xC3F8C004)) = 0xA1A11111; /* FLASH_LML */
    *((volatile uint32*)(0xC3F8C004)) = 0x00000000; /* FLASH_LML */
    *((volatile uint32*)(0xC3F8C00C)) = 0xC3C33333; /* FLASH_SLL */
    *((volatile uint32*)(0xC3F8C00C)) = 0x00000000; /* FLASH_SLL */
}
```


Chapter 6

Other Tresos Configuration

This chapter describes how to install the Tresos plugin into a new directory.

6.1 Setting up the plugins

For using an alternative plug-ins directory with EB tresos, please follow the steps below:

- Copy the plug-in(s) (named like `""Fls_TS_T2D13M10I1R0""`) into a directory `""<MyDirectoryPath>\<MyDirectoryName>""` under the folders `""<MyDirectoryPath>\<MyDirectoryName>\eclipse\plugins""`
- Create a text file with the extension `""link""` into the EB tresos installation in a folder called `""links""` (e.g. `<MyTresosIntallation>\links\<MyPluginPackageName>.link`)
- Put the following text in that `""<MyPluginPackageName>.link""` file:
`""path=<MyDirectoryPath>/<MyDirectoryName>""`, please make sure the path is described with forward slashes.

Chapter 7

Async/sync mode

7.1 Linear Address

The Fls driver provides services for reading, writing and erasing flash memory and it combines configured flash memory sectors into one linear address space.

The Fls module shall combine all available flash memory areas into one linear address space, it will always start at address 0 and continues without any gap.

Example:

If user wants to configure following sectors:

Table 7-1. Sectors details Example

FlsPhysicalSector	Fls Physical Start Address	Fls Sector Size
FLS_DFO_ARRAY_0_L00	0x00800000	0x4000
FLS_DFO_ARRAY_0_L02	0x00808000	0x4000

The FlsSector List should be configured in the following way:

In...	Name	X	Fls Physical Sector	Fls Sec...	Fls Se...
0	FlsSecto...	☒	FLS_DFO_ARRAY_0_L00	0x4000	0x0
1	FlsSecto...	☒	FLS_DFO_ARRAY_0_L02	0x4000	0x4000

Figure 7-1. Fls Sector List

As it can be seen physical address contains a gap in between the 2 sectors while the logical one doesn't; from the driver point of view, "Fls Sector Start Address" for FlsSector_0 will be 0x0 and "Fls Sector Start Address" for FlsSector_1 will be 0x4000.

If user wants to write in FLS_DFO_ARRAY_0_L02 sector, it is needed to write inside logical address [0x4000 - 0x7FFF].

If user wants to erase it, it is needed to erase sector from address 0x4000 with size 0x4000.

Note: It is not needed to calculate the "Fls Sector Start Address" and "Fls Sector Size" they can be automatically computed.

7.2 Fls Page Size vs Programming Size

The Page Size is the minimum size which can be programmed with one HV operation (generally it is a Double Word - 8 byte). On some platforms the Fls Page Size depends on the sector used (Page Size on Code Flash may be different from Page Size on Data flash), other platforms are able to manage different Page Size for the same sector, in this case we talk about Programming Size.

For **MPC560XB** Platform, Programming Size has the same value than Page Size and so the user has to select below option:

Code Flash:

- Fls Page size = 8
- Fls Programming Size = "FLS_WRITE_DOUBLE_WORD"

Data Flash:

- Fls Page size = 8
- Fls Programming Size = "FLS_WRITE_DOUBLE_WORD"

Data Flash Optimized (DFO on 5606B):

- Fls Page size = 4
- Fls Programming Size = "FLS_WRITE_WORD"

7.3 Async vs Sync

It's possible to modify the behavior of Fls sector erase / Fls page write using two configuration parameters (FlsPageEraseAsync, FlsPageWriteAsync) into FlsSector TAB.

If FlsSectorEraseAsynch/FlsPageWriteAsynch are enabled sector erase / page write job in the Fls_MainFunction are executed asynchronously, it means that Fls_MainFunction will not wait (not blocking) for completion of high voltage operation but at least 2 Fls_MainFunction are needed to complete the operation.

If FlsSectorEraseAsynch/FlsPageWriteAsynch are disabled sector erase / page write job in the Fls_MainFunction are executed synchronously, which means sector erase / page write job are blocking and any high voltage operation will be completed during one Fls_Mainfunction.

FlsSectorEraseAsynch Mode has been developed for embedded software which cannot be blocked for long time waiting the end of the HV operation.

Fls write in sync mode is similar to the async mode - In some cases the sync mode consumes less time to complete a Fls job (because less Fls_MainFunctions are needed), more information can be found in the FLS IM document.

7.4 Access Code Routine on Job Start

To avoid RWW (Read While Write) problems the flash driver provide the **FlsAcLoadOnJobStart** configuration parameter.

If it is enabled the Fls driver will load the **Access Code Routine** out of RAM when an erase or write job is started and unload (overwrite) it after that job has been completed or cancelled.

The Access Code Routine is used to set the MCR:EHV bit (start an HV operation) and wait for completion (bit MCR:DONE set), therefore loading this routine out of RAM will avoid any RWW problems.

When FlsAcLoadOnJobStart is disabled, the only way to avoid RWW problems is to use different flash array / partition from the one used by the application code or the application code should be out of RAM.

In case of Async operations it is only possible to erase / write to flash array different from flash array the application is executing from.

Note: The Access Code Routine is used only in sync mode (waiting for bit DONE - blocking mode) therefore in Async mode FlsAcLoadOnJobStart have NO effect at all.

7.5 Unlocking Fls Sectors

The Flash memory physical sectors that are going to be modified by Fls driver (i.e. erase and write operations) have to be unlocked for a successful operation.

If it is not handled by FLS driver must be setup on an application level. Unlock only those physical sectors that will be modified by Fls driver operation.

It is recommended to configure only those FlsPhysicalSector(s) that are required by upper layer module FEE. As FLS driver can access only those configured FlsPhysicalSector(s) and can not modify rest of Flash address space.

Note: If the microcontroller is in user mode, be sure that the Flash memory controller registers are accessible.

For more information please refer to the 'Memory Protection Unit' and 'Register Protection' chapters in the device reference manual.

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number UM13FLSASR4.0 Rev0003R1.0.1
Revision 3.0